

STAT DRP: Markov Chain Monte Carlo

Nidhi Vora

1 Introduction

A **Markov chain** is a mathematical process that models the probability of transitioning to a future state based solely on the current state, not on the history of how we arrived there. This is known as the *Memoryless Property*. **Monte Carlo** methods use repeated random sampling to approximate outcomes of uncertain processes. Together, **Markov Chain Monte Carlo (MCMC)** methods construct a Markov chain that, over many steps, produces samples from a target probability distribution π — even when that distribution is complex, high-dimensional, or known only up to a normalizing constant.

1.1 Example: Weather as a Markov Chain

Consider weather with two states: Sunny (S) and Rainy (R). Tomorrow's weather depends only on today's, satisfying the Markov property:

- If today is Sunny: $P(S \rightarrow S) = 0.8$, $P(S \rightarrow R) = 0.2$
- If today is Rainy: $P(R \rightarrow S) = 0.4$, $P(R \rightarrow R) = 0.6$

We encode these probabilities in a **transition matrix**:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

where row i is the current state and column j is the next state.

If today is Rainy, the initial state vector is $v_0 = [0 \quad 1]$. The probability distribution for tomorrow is:

$$v_1 = v_0 P = [0 \quad 1] \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} = [0.4 \quad 0.6]$$

So tomorrow there is a 40% chance of Sunny and 60% chance of Rainy. The distribution on day n is $v_n = v_0 P^n$. As $n \rightarrow \infty$, regardless of the starting state, v_n converges to a fixed **stationary distribution** π satisfying $\pi P = \pi$. This is the core idea behind MCMC: we design a Markov chain whose stationary distribution is exactly the target we want to sample from.

2 Metropolis-Hastings

2.1 Background

The Metropolis-Hastings (MH) algorithm generates samples from a target distribution $\pi(x)$ by repeatedly proposing a new state from a proposal distribution $q(x^* | x_t)$ and accepting or rejecting it via a probability ratio. Over many iterations, the resulting Markov chain converges to $\pi(x)$ as its stationary distribution, allowing us to approximate integrals and expectations without knowing the normalizing constant.

2.2 Algorithm

Starting from an initial value x_0 , at each step we draw a candidate x^* from the proposal $q(x^* | x_t)$ and accept it with probability:

$$\alpha(x_t, x^*) = \min \left(1, \frac{\pi(x^*) q(x_t | x^*)}{\pi(x_t) q(x^* | x_t)} \right)$$

The ratio $\pi(x^*)/\pi(x_t)$ favours moves to higher-density regions, while the correction term $q(x_t | x^*)/q(x^* | x_t)$ accounts for any asymmetry in the proposal. When the proposal is symmetric (e.g. a Gaussian centred at x_t), this correction equals 1 and the rule simplifies to comparing densities alone. Crucially, the normalizing constant of π cancels in the ratio, so we never need to compute it.

Algorithm 1: Metropolis-Hastings

Input: Target $\pi(x)$, Proposal $q(y | x)$, Initial x_0 , Iterations T

```

for  $t = 0$  to  $T - 1$  do
    Sample  $y \sim q(y | x_t)$  and  $u \sim \text{Uniform}(0, 1)$ ;
     $\alpha \leftarrow \min\left(1, \frac{\pi(y) q(x_t | y)}{\pi(x_t) q(y | x_t)}\right)$ ;
    if  $u \leq \alpha$  then
        |  $x_{t+1} \leftarrow y$ ;                                // Accept
    else
        |  $x_{t+1} \leftarrow x_t$ ;                            // Reject, stay
    end
end

```

2.3 Implementation

We sample from a standard normal $\mathcal{N}(0, 1)$ using a symmetric Gaussian proposal $q(x^* | x_t) = \mathcal{N}(x_t, 1)$, starting at $x_0 = 0$ with $T = 10,000$ iterations. The code samples from $\mathcal{N}(0, 1)$ by randomly proposing nearby values and accepting or rejecting them based on how likely they are under the target density.

```

1 import numpy as np
2
3 def metropolis_hastings(log_target, x0, n_samples, sigma=1.0):
4     x = x0
5     samples = []
6     for _ in range(n_samples):
7         x_star = np.random.normal(x, sigma)          # propose
8         log_alpha = log_target(x_star) - log_target(x) # log acceptance ratio
9         if np.log(np.random.rand()) < log_alpha:
10             x = x_star                                # accept
11             samples.append(x)
12     return np.array(samples)
13
14 # log of unnormalized N(0,1) density
15 log_target = lambda x: -0.5 * x**2
16
17 samples = metropolis_hastings(log_target, x0=0, n_samples=10000)
18 print(f"Mean: {samples.mean():.3f}, Std: {samples.std():.3f}")
19 # Output: Mean ~ 0.000, Std ~ 1.000

```

Running this yields a mean of approximately 0 and standard deviation of approximately 1, consistent with $\mathcal{N}(0, 1)$ [MRR⁺53, Has70].

3 Gibbs Sampling

3.1 Background

Gibbs sampling is a special case of Metropolis-Hastings designed for *multivariate* distributions where sampling from the full joint $\pi(\mathbf{x})$ is hard, but sampling from each **full conditional** — the distribution of one variable given all others — is tractable. By cycling through variables and sampling each from its conditional, the chain converges to the joint target. A key feature is that every proposal is always accepted ($\alpha = 1$), since drawing from the exact conditional automatically satisfies the conditions for stationarity.

3.2 Algorithm

Given a d -dimensional target $\pi(x_1, \dots, x_d)$, one full Gibbs iteration updates each component from its full conditional, using the most recently sampled values of all other variables:

$$x_i^{(t+1)} \sim \pi\left(x_i \mid x_1^{(t+1)}, \dots, x_{i-1}^{(t+1)}, x_{i+1}^{(t)}, \dots, x_d^{(t)}\right)$$

Algorithm 2: Gibbs Sampler

Input: Full conditionals $\pi(x_i \mid \mathbf{x}_{-i})$, Initial $\mathbf{x}^{(0)}$, Iterations T
for $t = 0$ **to** $T - 1$ **do**
 for $i = 1$ **to** d **do**
 Sample $x_i^{(t+1)} \sim \pi\left(x_i \mid x_1^{(t+1)}, \dots, x_{i-1}^{(t+1)}, x_{i+1}^{(t)}, \dots, x_d^{(t)}\right)$;
 end
end

3.3 Implementation

We sample from a bivariate normal with correlation $\rho = 0.8$:

$$\pi(x_1, x_2) = \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}\right)$$

The full conditionals are analytically available:

$$x_1 \mid x_2 \sim \mathcal{N}(\rho x_2, 1 - \rho^2), \quad x_2 \mid x_1 \sim \mathcal{N}(\rho x_1, 1 - \rho^2)$$

The code samples from the joint distribution by alternately drawing each variable from its conditional given the other, with no proposals ever rejected.

```

1 import numpy as np
2
3 def gibbs_bivariate_normal(rho, n_samples):
4     std = np.sqrt(1 - rho**2)
5     x1, x2 = 0.0, 0.0
6     samples = []
7     for _ in range(n_samples):
8         x1 = np.random.normal(rho * x2, std)    # sample x1 | x2
9         x2 = np.random.normal(rho * x1, std)    # sample x2 | x1
10        samples.append([x1, x2])
11    return np.array(samples)
12
13 samples = gibbs_bivariate_normal(rho=0.8, n_samples=10000)
14 print(f"Mean: {samples.mean(axis=0)}")
15 print(f"Covariance:\n{np.cov(samples.T)}")
16 # Mean ~ [0, 0], Cov ~ [[1, 0.8], [0.8, 1]]

```

The sample mean and covariance closely match the true values, confirming convergence to the target [GCS⁺13].

4 Discussion

Both algorithms construct a Markov chain that converges to a target distribution π , but they differ in when and how they are applicable.

Metropolis-Hastings is the more general method. It requires only the ability to evaluate $\pi(x)$ up to a constant and to sample from a proposal q . The trade-off is that proposals may be rejected, and a poorly chosen proposal can lead to slow mixing.

Gibbs sampling eliminates rejection entirely by sampling from exact conditionals, making every iteration productive. However, it requires those conditionals to be available in closed form, and it can mix slowly when variables are highly correlated, since it moves along one axis at a time.

In practice, the choice of algorithm depends on the structure of the problem: Gibbs is preferred when conditionals are tractable, while MH offers a flexible fallback for any target that can be evaluated pointwise.

5 Langevin-Based Algorithms

5.1 Background: Langevin Dynamics

Langevin dynamics offers a physically motivated way to sample from a target distribution $\pi(x) \propto e^{-U(x)}$, where $U(x)$ is called the **potential function**. The idea is to simulate a particle moving through a potential landscape: it is pulled toward regions of high probability (low potential) by a *drift* force, while random thermal fluctuations — modelled by Brownian motion — prevent it from settling at a single point.

Brownian motion. A standard Brownian motion $\{B_t\}_{t \geq 0}$ is a continuous random process with $B_0 = 0$ and independent increments $B_t - B_s \sim \mathcal{N}(0, t - s)$ for $t > s$. It serves as the mathematical model of pure random diffusion.

5.2 The Langevin SDE

The (overdamped) Langevin stochastic differential equation is:

$$dX_t = -\nabla U(X_t) dt + \sqrt{2} dB_t$$

- **Drift term** $-\nabla U(X_t) dt$: pushes the particle downhill in U , i.e. toward regions of higher probability under π .
- **Diffusion term** $\sqrt{2} dB_t$: injects random noise to ensure the particle continues to explore and does not collapse to the mode.

Under mild regularity conditions, this SDE has $\pi(x) \propto e^{-U(x)}$ as its unique **stationary distribution**: starting from any initial condition, X_t converges in distribution to π as $t \rightarrow \infty$.

5.3 Additional: Second-Order (Underdamped) Langevin

The **underdamped Langevin** equation augments the state with a momentum variable v_t :

$$dX_t = v_t dt, \quad dv_t = -\nabla U(X_t) dt - \gamma v_t dt + \sqrt{2\gamma} dB_t$$

where $\gamma > 0$ is a friction coefficient. The momentum allows the particle to carry information across the landscape, which can lead to faster mixing in practice, especially for distributions with multiple modes or strong correlations.

5.4 Additional: MALA

Metropolis-Adjusted Langevin Algorithm (MALA) corrects the discretization bias introduced by the Euler–Maruyama scheme by wrapping the Langevin proposal inside a Metropolis–Hastings accept/reject step.

$$x^* = x_k - \eta \nabla U(x_k) + \sqrt{2\eta} Z_k$$

but the candidate is accepted with probability $\min(1, \alpha)$ where:

$$\alpha = \frac{\pi(x^*) q(x_k | x^*)}{\pi(x_k) q(x^* | x_k)}, \quad q(y | x) \propto \exp\left(-\frac{\|y - x + \eta \nabla U(x)\|^2}{4\eta}\right)$$

MALA is **exact** (its stationary distribution is exactly π), at the cost of one additional density evaluation per step for the ratio.

5.5 Implementation: Gaussian Target

We apply ULA to a one-dimensional Gaussian target $\pi(x) = \mathcal{N}(\mu, \sigma^2)$. Writing $\pi(x) \propto e^{-U(x)}$, the potential and its gradient are:

$$U(x) = \frac{(x - \mu)^2}{2\sigma^2}, \quad \nabla U(x) = \frac{x - \mu}{\sigma^2}$$

The ULA update therefore becomes:

$$X_{k+1} = X_k - \eta \cdot \frac{X_k - \mu}{\sigma^2} + \sqrt{2\eta} Z_k, \quad Z_k \sim \mathcal{N}(0, 1)$$

We run $T = 15,000$ iterations with step size $\eta = 0.1$, targeting $\mathcal{N}(5.0, 2.0^2)$, and discard the first 10% as burn-in.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ula_gaussian(mu, sigma, step_size, n_samples, x0=0.0):
5     """
6     Samples from a 1D Gaussian using the Unadjusted Langevin Algorithm.
7     Target: pi(x) ~ exp(-(x-mu)^2 / (2*sigma^2))
8     Gradient of potential: grad U(x) = (x - mu) / sigma^2
9     """
10    samples = np.zeros(n_samples)
11    current_x = x0
12    noise_scale = np.sqrt(2 * step_size) # diffusion coefficient
13
14    for i in range(n_samples):
15        grad = (current_x - mu) / sigma**2 # drift
16        z = np.random.normal(0, 1) # noise
17        current_x = current_x - step_size * grad \
18                    + noise_scale * z # Euler-Maruyama step
19        samples[i] = current_x
20    return samples
21
22 # Target: N(5.0, 2.0^2)
23 TARGET_MU, TARGET_SIGMA = 5.0, 2.0
24 STEP_SIZE, N_ITER = 0.1, 15000
25
26 samples = ula_gaussian(TARGET_MU, TARGET_SIGMA, STEP_SIZE, N_ITER)
27 burn_in = int(N_ITER * 0.1)
28 final = samples[burn_in:]
29
30 # Plot histogram vs. true PDF
31 x_range = np.linspace(TARGET_MU - 4*TARGET_SIGMA,
32                        TARGET_MU + 4*TARGET_SIGMA, 200)
33 true_pdf = (np.exp(-0.5 * ((x_range - TARGET_MU)/TARGET_SIGMA)**2)
34             / (TARGET_SIGMA * np.sqrt(2 * np.pi)))
35
36 plt.figure(figsize=(10, 6))
37 plt.hist(final, bins=60, density=True, alpha=0.6,
38         color='skyblue', label='ULA Samples')
39 plt.plot(x_range, true_pdf, 'r--', lw=2, label='True PDF')
40 plt.title(f"ULA Sampling Results (Step Size={STEP_SIZE})")
41 plt.xlabel("Value"); plt.ylabel("Probability Density")
42 plt.legend(); plt.grid(axis='y', alpha=0.3); plt.show()

```

The histogram of post-burn-in samples closely matches the theoretical $\mathcal{N}(5.0, 4.0)$ density. Setting $\eta = 1.0$ causes the chain to diverge, while $\eta = 0.001$ produces extremely slow mixing, illustrating the step-size trade-off inherent to ULA.

6 Further Studies: Autoregressive models

Just as MCMC methods model the next sample as depending on the current state, autoregressive (AR) models are built on the same principle of sequential dependence. AR models predict future values of a variable using a linear combination of its own past values. The core idea is that the current value of a series depends on its previous observations, making it a natural framework for modeling temporal dependencies in sequential data.

In econometrics and finance, autoregressive models have long been used to forecast time series such as exchange rates, stock returns, and interest rates. They remain a foundational tool for capturing short-term momentum and mean-reverting behavior in financial data, and it is still a valid and widely used method in recent literature [RSVDH22].

Beyond classical statistics, the autoregressive structure also underlies modern large language models (LLMs), where each token is generated conditioned on all previously generated tokens. This sequential, one-step-ahead prediction framework is conceptually analogous to AR models in econometrics, despite operating in a vastly different domain [MYF⁺24].

References

- [GCS⁺13] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. CRC Press, 3rd edition, 2013.
- [Has70] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970.
- [MRR⁺53] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [MYF⁺24] R Thomas McCoy, Shunyu Yao, Dan Friedman, Mathew D Hardy, and Thomas L Griffiths. Embers of autoregression show how large language models are shaped by the problem they are trained to solve. *Proceedings of the National Academy of Sciences*, 121(41):e2322420121, 2024.
- [RSVDH22] Marta Regis, Paulo Serra, and Edwin R Van Den Heuvel. Random autoregressive models: A structured overview. *Econometric Reviews*, 41(2):207–230, 2022.